26 October 2026 23:13

```python
class Node:
    def __init__(self, key, val):
        # act as self.data
        self.key = key
        self.value = val

        self.next = None
        self.prev = None

```

```python
class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.cache = {}

        # The convention can be changed
        self.head = Node(0,0) # Least Recent
        self.tail = Node(0,0) # Most Recent
        self.head.next = self.tail
        self.tail.prev = self.head
```

{} (max length ≤ capacity)

None ← Node(0,0) ⇄ Node(0,0) → None

In this, our convention is
after head (least Recent accessed data)
before tail (Most Recently accessed data)

```python

    def remove(self, node):
        prev_node = node.prev
        next_node = node.next
        prev_node.next = next_node
        next_node.prev = prev_node

    def insert(self, node):
        prev_node = self.tail.prev
        next_node = self.tail
        prev_node.next = node
        next_node.prev = node
        node.next = next_node
        node.prev = prev_node

```

⇒ Remove or add/insert node other than head or tail
⇒ Remove from anywhere in bw head, tail
⇒ Insert only before tail ← based on convention

```python
    def get(self, key: int) -> int:
        if key in self.cache:
            node = self.cache[key]
            self.remove(node)
            self.insert(node)
            return node.value
        return -1

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            self.remove(self.cache[key])
        new_node = Node(key, value)
        self.cache[key] = new_node
        self.insert(new_node)
        if len(self.cache) > self.capacity:
            lru_node = self.head.next
            self.remove(lru_node)
            del self.cache[lru_node.key]
```

Input
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
Output
[null, null, null, 1, null, -1, null, -1, 3, 4]

Explanation
LRUCache lRUCache = new LRUCache(2);
lRUCache.put(1, 1); // cache is {1=1}
lRUCache.put(2, 2); // cache is {1=1, 2=2}
lRUCache.get(1);   // return 1
lRUCache.put(3, 3); // LRU key was 2, evicts key 2, cache is {1=1, 3=3}
lRUCache.get(2);   // returns -1 (not found)
lRUCache.put(4, 4); // LRU key was 1, evicts key 1, cache is {4=4, 3=3}
lRUCache.get(1);   // return -1 (not found)
lRUCache.get(3);   // return 3
lRUCache.get(4);   // return 4